Signal2Space — Deep-Learning Phase Handoff

Indoor positioning from Wi-Fi fingerprints · UJIIndoorLoc · Team: Krishiv, Gargi, Agrima, Abhiman, Bhawana
Repository: https://github.com/krishiv274/Signal2Space · Integration branch: `claude/peaceful-shannon-o1t6j8` · Planner: Notion task board

1. Status at a glance

Done and verified.

- PyTorch DL pipeline (`src/signal2space_dl/`, 6 modules) and a CLI. Twelve unit tests pass.
- Five notebooks (DL_01 to DL_05), executed with outputs saved.
- A **full end-to-end run** on CPU with seed 42: four models trained, about 4 minutes in total.
- All models evaluated on the same 1,111 held-out rows as AML, with a paired-bootstrap comparison.
- Updated README and `docs/DL_PIPELINE.md`; the Notion planner; five member ZIPs; this document and the viva scripts.

GitHub, done.

- Branch `claude/peaceful-shannon-o1t6j8` is pushed, and integration PR #7 is open with review requested.
- Tracker #6 is created, with sub-issues DL-0 #8, DL-1 #12, DL-2 #9, DL-3 #10, DL-4 #11, DL-5 #13 and DL-6 #14.
- Member branches DL-1 to DL-4 were created from the integration branch.
- Nothing is merged.

Open items.

- GitHub rejected `gigabyte2024` as an assignee and reviewer, most likely because Gargi is not a collaborator yet. DL-1 #12 is unassigned until Krishiv adds her.
- No labels or milestones were configured beyond those created automatically with the issues.
- Python 3.14 on macOS still needs verification in DL-0.

2. What Signal2Space does, and what existed before this phase

Signal2Space predicts a device's **building** (3 classes), **floor** (5 classes) and **2-D position** from received signal strengths of 520 Wi-Fi access points (WAPs). In UJIIndoorLoc, the value `100` means a WAP was not detected; about 96.5% of all entries are `100`. The data has 19,937 training/reference fingerprints and 1,111 validation fingerprints. The validation set was recorded later, by other users and phones, and is used as the held-out test set.

Phase 1 (AML), implemented and verified before this work.

- **ML_01.** A group split on building, floor and space (80/20, seed 42). RSSI scaling is fitted on internal-train rows only. Features are 520 scaled RSSI values, 520 presence flags and 6 summary statistics.
- **ML_02 to ML_04.** Random Forests for building, floor and coordinates, tuned on internal validation.
- **ML_05.** Integration and evaluation on held-out data.
- **Held-out results:** building 99.73%, floor 88.93%, coordinate MAE 8.10 / 6.81, mean Euclidean error 11.58.
- I recomputed these numbers from the committed row-level predictions, and they match the README.

At the start of this phase, the five DL notebooks were empty: valid JSON with no cells. `requirements.txt` had no PyTorch. In the cloud checkout used for this work, the git tree was clean. The untracked `src/signal2space_dl/pipeline.py` and the modified PPTX you described exist only on Krishiv's Mac. They were not visible here and were not modified. DL-0 tells Krishiv to preserve them and reconcile them. The new package deliberately has no `pipeline.py`, so checking out the branch cannot overwrite that file.

3. DL architecture and data flow

Data and leakage controls

1. Validate both CSVs: 529 columns in order, no blanks, numeric values, detected RSSI within [−110, 0] dBm.
2. Reproduce the ML_01 **group split** exactly: 16,085 internal-train and 3,852 internal-validation rows. The internal-validation `ROW_ID`s are verified identical to AML's.
3. Fit on internal-train rows only:
    - per-WAP mean and scale of detected readings;
    - a fill value for undetected WAPs, below the observed range;
    - label classes;
    - the coordinate centre.
4. Build the inputs: **1,040 features** = 520 standardised RSSI values + 520 presence flags. Dropout of 0.2 on the input simulates WAPs that a phone fails to detect.
5. Held-out data is only transformed, never used for fitting.

Network (765,834 parameters, multitask)

```
x[1040] → Dropout .2
 → Linear 512 → BN → ReLU → Dropout .3
 → Linear 256 → BN → ReLU → Dropout .3
   ├ building: 128 → 3 logits
   ├ floor:    128 → 5 logits
   └ coord:    128 → 2 (scaled lon, lat)
```

Loss: `CE(b) + CE(f) + Huber((xy − μ)/10)`, with fixed weights (1, 1, 1).

- Dividing coordinates by 10 makes a 10-unit error about as costly as a cross-entropy term. Z-scored coordinates would make the coordinate loss negligible.
- Huber loss bounds the gradients from large errors.
- The **single-task ablations** are the same network with only one head.

Training, selection and evaluation

- **Training:** AdamW (learning rate 1e-3, weight decay 1e-4), batch 256, ReduceLROnPlateau, at most 150 epochs.
- **Early stopping:** patience 15 on the internal-validation weighted loss; the best epoch is restored.
- **Reproducibility:** all random generators are seeded (42) and deterministic algorithms are enabled. The device is chosen automatically (CUDA, then MPS, then CPU).
- **No tuning:** hyperparameters were fixed in advance.
- **Frozen evaluation:** each checkpoint's SHA-256 is recorded at training time. Held-out evaluation refuses to run if the hash differs.

Model	Heads	Selected / run epochs	Parameters	CPU time	Checkpoint SHA-256
dl_02_building	building	13 / 28	699,139	30 s	`0b378063c94d`
dl_03_floor	floor	43 / 58	699,397	56 s	`b5e3dc0536dc`
dl_04_coordinate	coordinate	100 / 115	699,010	96 s	`2a084ffda258`
dl_05_multitask	building, floor, coordinate	48 / 63	765,834	60 s	`5f5df5088f8d`

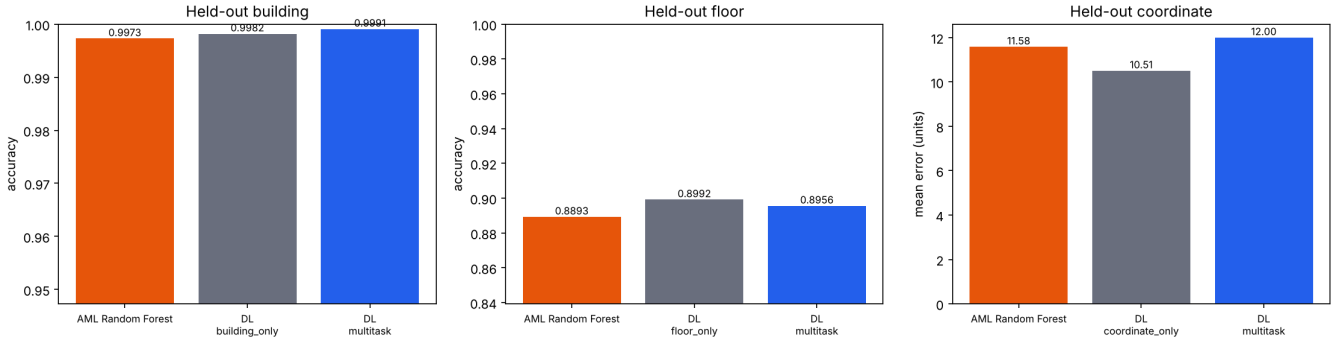
4. Results (full run) and comparison with AML

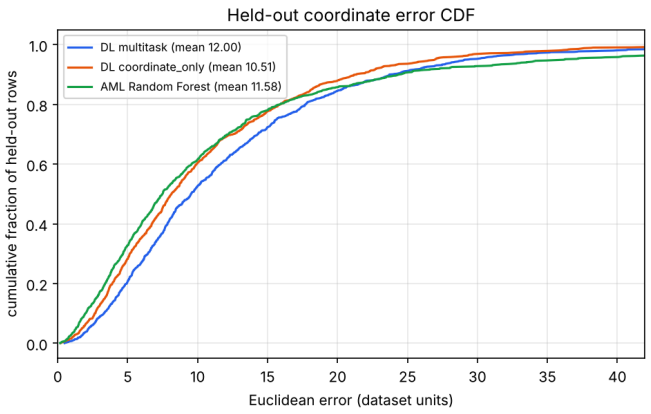
All rows are on the same held-out partition (`validationData.csv`, 1,111 rows) with the same metric functions. Coordinates are in dataset units; the EPSG:3857 label is unconfirmed.

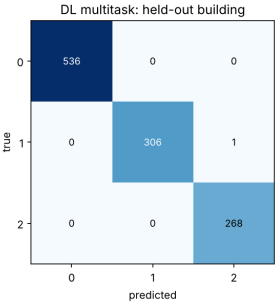
Held-out metric	AML Random Forest	DL single-task	DL multitask
Building accuracy	99.73%	99.82%	99.91%
Building macro F1	99.71%	99.83%	99.88%
Floor accuracy	88.93%	89.92%	89.56%
Floor macro F1	87.27%	88.43%	88.15%
Longitude MAE	8.10	7.24	8.62
Latitude MAE	6.81	6.11	6.72
Mean Euclidean error	11.58	10.51	12.00
Median Euclidean error	7.49	8.13	9.53
p90 Euclidean error	24.39	21.35	23.71
RMSE (Euclidean)	17.67	13.46	15.02
Joint building + floor correct	986 / 1,111	—	995 / 1,111

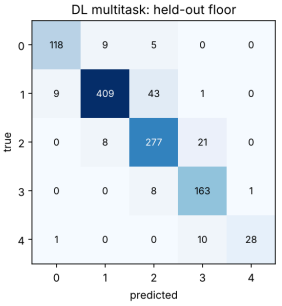
Paired bootstrap: DL minus AML on identical rows (2,000 resamples)

DL model	Metric	Difference	95% CI	Reading
DL building_only	building accuracy	+0.09 pp	[-0.18, +0.36]	within noise
DL floor_only	floor accuracy	+0.99 pp	[-0.36, +2.43]	within noise
DL coordinate_only	mean euclidean error	-1.08	[-1.78, -0.37]	resolved
DL multitask	building accuracy	+0.18 pp	[-0.18, +0.54]	within noise
DL multitask	floor accuracy	+0.63 pp	[-0.72, +1.89]	within noise
DL multitask	mean euclidean error	+0.42	[-0.33, +1.16]	within noise









Interpretation

- **Building** is saturated for both model families. The differences are 1–2 rows.
- **Floor:** DL is 0.6–1.0 percentage points higher, but within noise. Internal-validation floor accuracy is about 97% while held-out is about 90%, and AML shows the same gap, which indicates distribution shift. Floor 4 is the smallest class and has the lowest held-out recall (72–74%).
- **Coordinates:** the **coordinate-only** DL model is the only resolved improvement over AML: mean error is lower by 1.08 (CI −1.78 to −0.37), and p90 and RMSE are also lower. AML keeps the lower median, so DL mainly reduces large errors.
- **Multitask:** sharing the trunk made coordinate error worse (12.00 vs 10.51). Building accuracy and joint building-and-floor correctness are slightly higher (995 vs 986 for AML).
- All models were frozen before held-out scoring. Choosing a per-task combination now would be post-hoc. The next step is to tune the coordinate weight on internal validation only, then score once.

5. Team plan, issue and PR sequence

#	Member	GitHub	Role	Issues	Handoff ZIP
1	**Krishiv**	`krishiv274`	Integration owner	DL-0 #8, DL-5 #13, DL-6 #14	Signal2Space_DL_handoff_1_Krishiv_krishiv274.zip
2	**Gargi**	`gigabyte2024`	Data & leakage audit (DL_01)	DL-1 #12	Signal2Space_DL_handoff_2_Gargi_gigabyte2024.zip
3	**Agrima**	`slayplayagrima`	Building head & robustness (DL_02)	DL-2 #9	Signal2Space_DL_handoff_3_Agrima_slayplayagrima.zip
4	**Abhiman**	`Koray-afk`	Floor head & class imbalance (DL_03)	DL-3 #10	Signal2Space_DL_handoff_4_Abhiman_Koray-afk.zip
5	**Bhawana**	`bhawanaydv`	Coordinate head & CRS/units (DL_04)	DL-4 #11	Signal2Space_DL_handoff_5_Bhawana_bhawanaydv.zip

Step	ID	Owner	Task	Branch	Depends on	Start
1	**DL-0** #8	Krishiv (krishiv274)	Review, verify on macOS and merge the DL integration PR	`claude/peaceful-shannon-o1t6j8`	—	Start now
2	**DL-1** #12	Gargi (gigabyte2024)	Data & leakage audit: real-data tests for DL_01 split and preprocessing	`dl/01-data-audit-gigabyte2024`	DL-0	Start now on your pre-created branch; open the PR into main after DL-0 merges; merges first in step 2
2	**DL-2** #9	Agrima (slayplayagrima)	Building head: error review and missing-AP robustness (DL_02)	`dl/02-building-robustness-slayplayagrima`	DL-0	Start now on your pre-created branch, in parallel with DL-1, DL-3 and DL-4; open the PR after DL-0 merges; merge after DL-1
2	**DL-3** #10	Abhiman (Koray-afk)	Floor head: per-building errors and class-weighting ablation (DL_03)	`dl/03-floor-imbalance-Koray-afk`	DL-0	Start now on your pre-created branch, in parallel with DL-1, DL-2 and DL-4; open the PR after DL-0 merges; merge after DL-1

2	**DL-4** #11	Bhawana (bhawanaydv)	Coordinate head: error analysis and CRS/unit verification (DL_04)	`dl/04-coordinate-crs-bhawanaydv`	DL-0	Start now on your pre-created branch, in parallel with DL-1, DL-2 and DL-3; open the PR after DL-0 merges; merge after DL-1
3	**DL-5** #13	Krishiv (krishiv274)	Final integration: re-run, reconcile results, update report/PPT, tag release	`dl/05-integration-krishiv274`	DL-1, DL-2, DL-3, DL-4	Start after DL-1 to DL-4 are merged
4	**DL-6** #14	All five (Krishiv coordinates)	Viva recordings (≤5 min each) for all five members	`(no branch; links are posted as issue comments)`	DL-5	Rehearse any time; record after DL-5 so the numbers are final

Start now: all five members. The member branches already contain the DL code; only opening and merging member PRs waits for DL-0.

Merge order:

1. DL-0: PR #7, with a merge commit, not squash.
2. DL-1, so the data guarantees are tested first.
3. DL-2, DL-3 and DL-4 in any order, each rebased on `main`.
4. DL-5 (re-run, reconcile, report and PPT, tag `v2.0-dl`).
5. DL-6 (vivas).

Conventions:

- Branch names follow `dl/<step>-<topic>-<github-id>`.
- Commits use `DL-n: … (refs #issue)`.
- PRs into `main` contain `Closes #issue` and request review from `krishiv274`.
- Never force-push `main` or retrain committed checkpoints in place.
- Never change a model based on held-out results.

What each member does:

- **Krishiv** integrates and validates the combined work (DL-0, DL-5, DL-6) and does not re-implement member work.
- **Gargi** proves the data guarantees with tests on the real data.
- **Agrima** stress-tests building prediction against missing access points.
- **Abhiman** analyses floor imbalance and tests class weighting with a decision rule fixed in advance.
- **Bhawana** analyses coordinate error and resolves the coordinate units/CRS question.

6. Links

- Repository: https://github.com/krishiv274/Signal2Space. Integration PR: #7. Branch: `claude/peaceful-shannon-o1t6j8`.
- Notion planner: https://app.notion.com/p/415e7fb36c6881d9b60effca159d11d5
- Issues: tracker #6; DL-0 #8, DL-1 #12, DL-2 #9, DL-3 #10, DL-4 #11, DL-5 #13, DL-6 #14.
- Member ZIPs, in `handoff/` on the integration branch:
  - Signal2Space_DL_handoff_1_Krishiv_krishiv274.zip
  - Signal2Space_DL_handoff_2_Gargi_gigabyte2024.zip

- Signal2Space_DL_handoff_3_Agrima_slayplayagrima.zip
- Signal2Space_DL_handoff_4_Abhiman_Koray-afk.zip
- Signal2Space_DL_handoff_5_Bhawana_bhawanaydv.zip

7. Reproducibility

From a clean clone, run:

1. `python3 -m venv .venv`
2. `source .venv/bin/activate`
3. `python -m pip install -r requirements-dl.txt` — this is the unchanged AML lock plus `torch==2.15.0` and `pytest==9.2.0`.
4. `python -m pytest -q tests`
5. `python scripts/run_dl_pipeline.py all` to retrain everything (about 4 minutes on CPU), or `evaluate` to re-score the committed checkpoints.

The notebooks DL_01 to DL_05 run in order and reuse the checkpoints by default.

Verified environment: Python 3.13, Linux, CPU, torch 2.15.0, NumPy 2.5.4, pandas 3.0.6, scikit-learn 1.9.1. Python 3.14 on macOS is pending verification in DL-0. Retraining on other hardware can change the last digits.

8. Limitations

- **Coordinate units/CRS are unconfirmed.** The values look like EPSG:3857 metres; Web-Mercator also overstates ground distance by about 1/cos(latitude). DL-4 is assigned to check this.
- **The supplied split** does not establish performance at unseen positions, in unseen buildings, or on unseen devices in general.
- **One seed, no hyperparameter search.** Seed variance is unmeasured.
- **Class weighting** is implemented but was off for the reported run; DL-3 evaluates it.
- **Not implemented:** learned loss weighting, building-conditioned floor heads, augmentation beyond input dropout.

Viva script — Krishiv (`krishiv274`)

Focus: Project overview, DL architecture, integration and final AML comparison · **Planned length:** 4:55 (limit 5:00) · Placeholders in [brackets] are filled from the member's own results before recording.

Time	Section	What to say	What to show on screen
0:00–0:40	**Problem and data**	Signal2Space predicts building, floor and position from Wi-Fi fingerprints in UJIIndoorLoc: 520 access points, where 100 means not detected. We have about 20,000 training fingerprints and 1,111 held-out ones.	README top section; `data/figures/ml_01_coordinate_labels_eda.png`
0:40–1:20	**Phase 1 recap**	Phase 1 was Random Forests with a group split. On held-out data they reached 99.73% building, 88.93% floor and 11.58 units mean coordinate error.	`data/tables/ml_05_aml_summary_metrics.csv`; README results table
1:20–2:30	**DL architecture**	Each fingerprint becomes 1,040 inputs: 520 standardised strengths and 520 'was it heard' flags. A shared two-layer trunk feeds three heads. The loss adds two cross-entropies and a Huber loss on coordinates divided by 10, which keeps the three terms on a similar scale.	`src/signal2space_dl/model.py` (MultitaskMLP, MultitaskLoss); `docs/DL_PIPELINE.md` §4 diagram
2:30–3:30	**Integration and discipline**	My job was integration. DL uses exactly the AML split; we verify the row IDs match. Every model is frozen on internal validation, and evaluation refuses to run if the checkpoint hash changed.	`evaluate.py::evaluate_experiment` (SHA check); `dl_01_split_parity.json`; GitHub PR for DL-0 and the tracker issue
3:30–4:30	**Results vs AML**	Multitask: 99.91% building, 89.56% floor, 995 of 1,111 rows with both correct, against 986 for AML. The coordinate-only network is the only resolved win: mean error 10.51 against 11.58. Sharing the trunk hurt coordinates.	`DL_05` notebook comparison table; `dl_05_heldout_model_comparison.png`; `dl_05_vs_aml_paired_bootstrap.csv`
4:30–4:55	**Limits and close**	These hold for the supplied split only. The CRS is unconfirmed and we used one seed. Next steps are a multi-seed run and tuning the	`docs/DL_PIPELINE.md` §9

		coordinate weight on internal validation.	

Contribution in one line: Review, verify and merge the integration PR #7, which holds the implemented DL pipeline (package, notebooks, full-run outputs, docs and handoff). Verify it on the team's macOS / Python 3.14 environment and preserve existing local work. This sits in one pipeline: DL_01 (data) → DL_02–DL_04 (heads) → DL_05 (multitask and the AML comparison).

Viva script — Gargi (`gigabyte2024`)

Focus: Data preparation, leakage controls and the data-audit tests · **Planned length:** 4:40 (limit 5:00) · Placeholders in [brackets] are filled from the member's own results before recording.

Time	Section	What to say	What to show on screen
0:00–0:35	**My role**	I own DL_01: turning the raw CSVs into network inputs without leaking held-out information.	DL-1 issue on GitHub; `DL_01_data_preparation.ipynb` title cell
0:35–1:35	**Schema and missing signals**	We validate all 529 columns. About 96.5% of signal values are 100, which means not detected. We fill those below the weakest real reading and add a presence flag, so the network can tell 'absent' from 'weak'.	DL_01 section 1 output (data-quality table); `data.py::validate_schema`, `Preprocessor.transform`
1:35–2:40	**Split and leakage control**	We group by building, floor and space, so a reference space never sits in both internal train and validation. The split is identical to AML's, row for row. Every statistic is fitted on the 16,085 internal-train rows only.	DL_01 section 2 (split summary, parity = true); `data.py::make_group_split`
2:40–3:50	**Audit tests**	My tests run on the real files and fail if any of these guarantees break: row counts, no shared groups, matching row IDs, and fitted parameters untouched by held-out data.	`tests/test_dl_data_real.py`, run with `pytest -q`; your DL-1 PR
3:50–4:40	**Findings and link to the project**	I also counted identical fingerprints across partitions [quote your number]. That is why our results describe this split, not unseen locations. Every later notebook relies on this preparation.	`dl_01_fingerprint_overlap.csv`; `dl_01_input_representation.png`

Contribution in one line: Prove, with automated tests on the real CSVs, that the DL split and preprocessing are leakage-free and match AML. Document the audit. This sits in one pipeline: DL_01 (data) → DL_02–DL_04 (heads) → DL_05 (multitask and the AML comparison).

Viva script — Agrima (`slayplayagrima`)

Focus: Building classification head and robustness to missing access points · **Planned length:** 4:35 (limit 5:00) · Placeholders in [brackets] are filled from the member's own results before recording.

Time	Section	What to say	What to show on screen
0:00–0:35	**My role**	I own DL_02, the building head, and its robustness analysis.	DL-2 issue; `DL_02_building_prediction.ipynb`
0:35–1:35	**Model and training**	It is the shared network with just the building head: three logits and cross-entropy. Early stopping on internal validation picked epoch 13 of 28.	`model.py`; `dl_02_building_training_curves.png`; `models/dl_02_building_model.json`
1:35–2:35	**Held-out result**	99.82% held-out accuracy against 99.73% for the Random Forest. That is one row, and the bootstrap interval includes zero, so building is essentially solved by both.	DL_02 metrics and AML comparison cells; `dl_02_building_heldout_confusion_matrix.png`
2:35–3:15	**Error review**	Both errors are building 0 predicted as building 1. [Quote the phone and user IDs and WAP counts from your table.]	Your error-review table in DL_02
3:15–4:35	**Robustness**	I hid 10 to 50% of the detected access points on internal validation and measured accuracy. [Quote the curve.] This tells us how the model behaves when a phone hears fewer networks.	`dl_02_building_ap_masking_robustness.png`; `evaluate.py::mask_detected_aps`; your DL-2 PR

Contribution in one line: Explain the building model's remaining errors and measure how accuracy degrades when detected access points go missing. This sits in one pipeline: DL_01 (data) → DL_02–DL_04 (heads) → DL_05 (multitask and the AML comparison).

Viva script — Abhiman (`Koray-afk`)

Focus: Floor classification head, class imbalance and the weighting ablation · **Planned length:** 4:40 (limit 5:00) · Placeholders in [brackets] are filled from the member's own results before recording.

Time	Section	What to say	What to show on screen
0:00–0:35	**My role**	I own DL_03, floor prediction, the hardest classification task.	DL-3 issue; `DL_03_floor_prediction.ipynb`
0:35–1:35	**Model and the gap**	The floor-only network reaches 97.4% on internal validation but 89.92% on held-out. AML shows the same gap, so the held-out data, recorded later with other phones, is genuinely harder.	`dl_03_floor_training_curves.png`; DL_03 metrics cell
1:35–2:35	**Where errors happen**	Floor 4 exists only in building 2 and has the lowest recall, 74% on 39 rows. Many floor-1 rows are predicted as floor 2. [Quote your by-building table.]	`dl_03_floor_heldout_confusion_matrix.png`; `dl_03_floor_heldout_per_class.csv`; `dl_03_floor_errors_by_building.csv`
2:35–3:50	**Class-weighting ablation**	I fixed the rule first: adopt weighting only if internal-validation macro F1 rises by at least 0.5 points. [State the result and decision.] Held-out was scored once afterwards.	Your DL-3 PR; weighted vs unweighted table in DL_03
3:50–4:40	**Comparison and link**	Unweighted DL floor is 89.92% against AML's 88.93%. The interval includes zero, so this is not a proven improvement. Floor errors also drive the larger coordinate errors in the hierarchy analysis.	`dl_05_hierarchical_breakdown.csv`; `dl_05_vs_aml_paired_bootstrap.csv`

Contribution in one line: Floor is the weakest classification task (held-out 89.9%; floor 4 recall 74%). Quantify where the errors happen and test class weighting with a decision rule fixed in advance. This sits in one pipeline: DL_01 (data) → DL_02–DL_04 (heads) → DL_05 (multitask and the AML comparison).

Viva script — Bhawana (`bhawanaydv`)

Focus: Coordinate regression head, error analysis and CRS/units · **Planned length:** 4:40 (limit 5:00) · Placeholders in [brackets] are filled from the member's own results before recording.

Time	Section	What to say	What to show on screen
0:00–0:35	**My role**	I own DL_04, coordinate regression, and the question of what the coordinate units really are.	DL-4 issue; `DL_04_coordinate_prediction.ipynb`
0:35–1:35	**Model and loss**	The coordinate head outputs two numbers. Targets are centred and divided by 10, and the loss is Huber, so errors above about 10 units do not dominate the gradients.	`model.py::MultitaskLoss`; `dl_04_coordinate_training_curves.png`
1:35–2:45	**Result vs AML**	The coordinate-only network gets a mean error of 10.51 against 11.58 for the Random Forest. The bootstrap interval excludes zero. The DL tail is better (p90 21.3 against 24.4), but the Random Forest has the lower median.	DL_04 comparison cell; `dl_05_heldout_error_cdf.png`; `dl_05_vs_aml_paired_bootstrap.csv`
2:45–3:35	**Where error happens**	[Quote your by-building and by-floor table.] Rows whose floor is predicted wrong have roughly double the coordinate error.	`dl_04_coordinate_error_by_building_floor.csv`; `dl_04_coordinate_errors.png`
3:35–4:40	**CRS verification**	Converting from Web-Mercator, the points land [state result]. Web-Mercator inflates distances by about 1/cos(latitude), so ground error is about 0.77 times the reported numbers. We report dataset units and label this as an estimate.	Your CRS note in DL_04 and `docs/DL_PIPELINE.md` §9; your DL-4 PR

Contribution in one line: Explain where coordinate error comes from and resolve the open question of coordinate units and CRS, without changing the reported metrics. This sits in one pipeline: DL_01 (data) → DL_02–DL_04 (heads) → DL_05 (multitask and the AML comparison).